

REACT JS – DAY 1 NOTES

1. What is React?

React is an open-source JavaScript library used to build **interactive and reusable user interfaces (UI)**, especially for web applications.

- Created at Facebook
- Open-sourced in **2013**
- Maintained by Meta and the open-source community
- Mainly focuses on the **UI layer**
- Uses a **declarative** and **component-based** approach

Basic Idea

Traditional JavaScript:

```
Select DOM
↓
Change DOM
↓
Update UI
```

React:

```
Data / State
↓
Describe UI
↓
React renders
↓
UI
```

Simple Example

```
function App() {
  return <h1>Hello React</h1>;
}

export default App;
```

2. Why Do We Use React?

Large applications may contain:

```
Navbar  
Sidebar  
Dashboard  
Profile  
Products  
Cart  
Footer
```

Instead of putting everything into one huge file, React divides the application into **components**.

```
Application  
|  
├─ Navbar  
├─ Sidebar  
├─ Dashboard  
│   ├── Card  
│   ├── Chart  
│   └── Table  
└─ Footer
```

Benefits

- Easier development
- Reusability
- Better organization
- Easier testing
- Easier maintenance
- Better readability

3. Important Features of React

3.1 Component-Based Architecture

React applications are built using components.

```
function Header() {  
  return <h1>My Website</h1>;  
}
```

Use the component:

```
<Header />
```

3.2 Declarative UI

React lets us describe **what the UI should look like** instead of manually describing every DOM operation.

```
function App() {  
  const name = "Arun";  
  
  return <h1>Welcome {name}</h1>;  
}
```

Remember

```
Data  
↓  
Describe UI  
↓  
React handles rendering
```

3.3 JSX

JSX allows **HTML-like markup inside JavaScript**.

```
const element = <h1>Hello React</h1>;
```

3.4 Virtual DOM

React uses an **in-memory representation of the UI** as part of its rendering process.

React determines the necessary changes before committing updates to the browser DOM.

3.5 Reusable Components

Create once:

```
function Button() {  
  return <button>Login</button>;  
}
```

Reuse:

```
<Button />  
<Button />  
<Button />
```

3.6 One-Way Data Flow

React commonly passes data from **parent** → **child** using props.

```
Parent  
  ↓  
Props  
  ↓  
Child
```

3.7 State-Based UI

When component state changes, React can update the UI.

```
State Changes  
  ↓  
Component Renders Again  
  ↓  
React Calculates Required Changes
```

↓
Browser UI Updated

4. React Application Basic Flow

```
Browser
  ↓
index.html
  ↓
main.jsx
  ↓
<App />
  ↓
Components
  ↓
JSX
  ↓
React Rendering
  ↓
Browser DOM
  ↓
UI
```

5. Creating a React Project with Vite

Vite is a common modern setup for React projects.

```
npm create vite@latest
```

Select:

```
Project Name → my-react-app
Framework → React
Variant → JavaScript
```

Then:

```
cd my-react-app
npm install
npm run dev
```

Important Commands

Command	Purpose
<code>npm create vite@latest</code>	Create Vite project
<code>cd my-react-app</code>	Enter project folder
<code>npm install</code>	Install dependencies
<code>npm run dev</code>	Start development server

6. React Project Structure

Typical Vite React project:

```
my-react-app/
|
├─ node_modules/
├─ public/
├─ src/
|   ├─ assets/
|   ├─ App.jsx
|   ├─ App.css
|   ├─ index.css
|   └─ main.jsx
├─ index.html
├─ package.json
└─ vite.config.js
```

Beginner Important Files

```
index.html
main.jsx
App.jsx
```

7. index.html

`index.html` is the HTML entry page loaded by the browser.

It contains the root element:

```
<div id="root"></div>
```

React mounts the application inside this element.

Easy Memory

```
HTML
↓
Empty #root
↓
React builds application inside it
```

8. main.jsx

`main.jsx` is commonly the **JavaScript entry point** of a Vite React application.

Example:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Important Part

```
createRoot(document.getElementById("root"))
```

It finds:

```
<div id="root"></div>
```

Then:

```
.render(<App />);
```

starts rendering the React application.

Connection

```
index.html
  ↓
#root
  ↓
main.jsx
  ↓
createRoot()
  ↓
<App />
  ↓
Application UI
```

9. App.jsx

`App.jsx` is commonly used as the **root application component** in a Vite React project.

Example:

```
function App() {
  return (
    <div>
      <h1>Welcome to React</h1>
      <p>My First Application</p>
    </div>
  );
}
```

```
export default App;
```

Important

`App.jsx` is **not a mandatory React filename**.

It is simply a common project convention.

Think

```
main.jsx
  ↓
App.jsx
  ↓
Other Components
```

10. Component

A **component** is a reusable piece of UI.

Modern React components are commonly JavaScript functions that return JSX.

Normal Function

```
function Welcome() {
  return (
    <h1>Hello Student</h1>
  );
}

export default Welcome;
```

Arrow Function

```
const Welcome = () => {
  return (
    <h1>Hello Student</h1>
  );
};
```

```
export default Welcome;
```

11. Component Naming Rule

Component names should start with a **capital letter**.

Correct:

```
function Header() {  
  return <h1>Header</h1>;  
}
```

Use:

```
<Header />
```

React distinguishes custom components from built-in/host elements using capitalization.

Built-in Elements

```
<div>  
<button>  
<input>
```

Custom Components

```
<Header />  
<Navbar />  
<Footer />
```

12. Why Components Are Important

Without components:

One Huge File

```
├─ Navbar
├─ Banner
├─ Products
├─ Cart
└─ Footer
```

With components:

App

```
|
├─ Navbar
├─ Banner
├─ ProductList
|   ├─ ProductCard
|   ├─ ProductCard
|   └─ ProductCard
├─ Cart
└─ Footer
```

Benefits

- Reusability
- Maintainability
- Readability
- Separation of concerns
- Easier testing
- Better application structure

13. Creating Multiple Components

Folder structure:

```
src/
├─
├─ components/
|   ├─ Header.jsx
|   ├─ Content.jsx
|   └─ Footer.jsx
├─
└─
```

```
├─ App.jsx
└─ main.jsx
```

Header.jsx

```
const Header = () => {
  return (
    <header>
      <h1>My React App</h1>
    </header>
  );
};

export default Header;
```

Content.jsx

```
const Content = () => {
  return (
    <main>
      <h2>Welcome</h2>
      <p>Learning React is fun!</p>
    </main>
  );
};

export default Content;
```

Footer.jsx

```
const Footer = () => {
  return (
    <footer>
      <p>Copyright 2026</p>
    </footer>
  );
};

export default Footer;
```

14. Import Components into App.jsx

```
import Header from "../components/Header";
import Content from "../components/Content";
import Footer from "../components/Footer";

const App = () => {
  return (
    <div>
      <Header />
      <Content />
      <Footer />
    </div>
  );
};

export default App;
```

Execution Flow

```
main.jsx
  ↓
<App />
  ↓
App.jsx
  ↓
Header
Content
Footer
  ↓
Complete UI
```

15. JSX

JSX = JavaScript XML

JSX is a syntax extension commonly used with React that allows HTML-like markup to be written inside JavaScript.

Example:

```
const App = () => {  
  return (  
    <h1>Hello React</h1>  
  );  
};
```

This:

```
<h1>Hello React</h1>
```

is JSX.

Important

Browsers do not directly execute JSX as plain JavaScript.

Build tools such as Vite transform JSX into JavaScript that browsers can execute.

16. JavaScript Inside JSX

Use curly braces:

```
{ }
```

Example:

```
const App = () => {  
  
  const name = "Arun";  
  const age = 22;  
  
  return (  
    <div>  
      <h1>Hello {name}</h1>  
      <p>Age: {age}</p>  
    </div>  
  );  
};
```

Output:

```
Hello Arun  
Age: 22
```

Easy Memory

```
JSX  
+  
{ JavaScript Expression }
```

17. JSX Rules

Rule 1: Single Parent

Incorrect:

```
return (  
  <h1>Hello</h1>  
  <p>Welcome</p>  
);
```

Correct:

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </div>  
);
```

Or use Fragment

```
return (  
  <>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </>  
);
```

18. className

HTML uses:

```
class="title"
```

JSX uses:

```
className="title"
```

Remember

```
HTML → class  
JSX  → className
```

19. Self-Closing Elements

Examples:

```
  
  
<input type="text" />  
  
<br />
```

20. Event Syntax

HTML:

```
<button onclick="showData()">
```

React:

```
<button onClick={showData}>
  Click
</button>
```

Example

```
const App = () => {

  const showMessage = () => {
    alert("Hello React");
  };

  return (
    <button onClick={showMessage}>
      Click Me
    </button>
  );
};

export default App;
```

Remember

```
HTML → onclick
React → onClick
```

21. Real DOM

DOM = Document Object Model

The browser represents an HTML document as a tree of nodes/elements.

HTML:

```
<div>
  <h1>Hello</h1>
  <p>Welcome</p>
</div>
```

Conceptual DOM:

```
div
├─ h1
│   └─ Hello
└─ p
    └─ Welcome
```

The browser-managed tree is commonly called the **Real DOM**.

22. Traditional DOM Update

HTML:

```
<h1 id="count">0</h1>

<button id="btn">
  Add
</button>
```

JavaScript:

```
let count = 0;

const heading = document.getElementById("count");
const button = document.getElementById("btn");

button.addEventListener("click", () => {
  count++;
  heading.textContent = count;
});
```

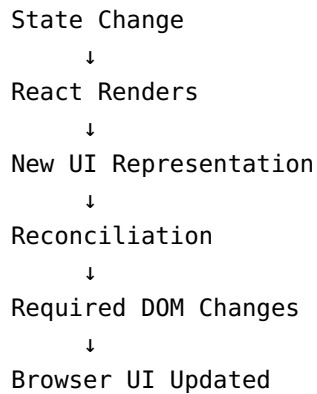
Here, JavaScript manually:

```
Find Element
  ↓
Listen for Event
  ↓
Update Element
```

23. Virtual DOM

Virtual DOM is a common term for React's **in-memory representation of the UI**.

When state changes:



24. Virtual DOM vs Real DOM

Real DOM	Virtual DOM Concept
Browser's actual DOM tree	React's in-memory UI representation
Browser APIs interact with it	Managed internally by React
Actual page changes happen here	Helps React determine desired changes
Browser rendering work occurs here	React determines changes before commit

Interview Clarification

Do **not** simply say:

React is fast because Virtual DOM is always faster than DOM.

Better:

React provides a declarative rendering model and determines the necessary UI updates for us. Performance depends on the application.

25. Reconciliation

Reconciliation is the process React uses to determine how a newly rendered UI relates to the previous UI and what updates are necessary.

When props or state change:

```
Previous UI
  ↓
State Changes
  ↓
New Render
  ↓
Reconciliation
  ↓
Determine Differences
  ↓
Commit Necessary Updates
  ↓
Real DOM
```

26. Reconciliation Example

Initial:

```
<h1>Count: 0</h1>
<p>Welcome</p>
```

After update:

```
<h1>Count: 1</h1>
<p>Welcome</p>
```

The relevant change is:

```
Count: 0
  ↓
Count: 1
```

React determines the required update and commits the corresponding DOM change.

27. State Update Example

```
import { useState } from "react";

const App = () => {

  const [count, setCount] = useState(0);

  const increase = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <h1>Count: {count}</h1>

      <button onClick={increase}>
        Add
      </button>
    </div>
  );
};

export default App;
```

Initial:

```
count = 0
```

Click:

```
setCount(1)
```

Conceptually:

```
State Changes
  ↓
Component Scheduled to Render
  ↓
```

```
App() Runs Again
  ↓
New JSX Result
  ↓
Reconciliation
  ↓
Required Change Determined
  ↓
DOM Commit
  ↓
Count: 1
```

28. React Rendering Process

VERY IMPORTANT

```
USER ACTION
  ↓
EVENT
  ↓
STATE UPDATE
  ↓
RENDER
  ↓
RECONCILIATION
  ↓
COMMIT
  ↓
BROWSER UI UPDATE
```

Example:

```
Click Button
  ↓
onClick
  ↓
setCount()
  ↓
Component renders
  ↓
React reconciles
  ↓
Necessary DOM update
```

↓
New value visible

29. React vs Traditional JavaScript

Vanilla JavaScript

```
const title = document.getElementById("title");  
  
title.textContent = "Welcome";
```

We explicitly say:

Find this element
↓
Change this property

React

```
const App = () => {  
  
  const name = "Welcome";  
  
  return (  
    <h1>{name}</h1>  
  );  
};
```

React's style is more **declarative**:

Data / State
↓
Describe UI
↓
React Rendering
↓
DOM Updates

30. return in a Component

A component commonly uses `return` to describe what should appear in the UI.

```
const Header = () => {  
  
  return (  
    <h1>Welcome</h1>  
  );  
  
};
```

Think

```
Component Function  
  ↓  
  return  
    ↓  
    JSX  
      ↓  
React renders UI
```

31. export default

Create component:

```
const Header = () => {  
  return <h1>Header</h1>;  
};
```

Export:

```
export default Header;
```

Import:

```
import Header from './Header';
```

Use:

```
<Header />
```

Flow

```
Header.jsx  
  ↓  
  export  
  ↓  
App.jsx  
  ↓  
  import  
  ↓  
<Header />
```

32. CSS in React

App.css

```
.title {  
  color: blue;  
  font-size: 40px;  
}
```

App.jsx

```
import './App.css';  
  
const App = () => {  
  
  return (  
    <h1 className="title">  
      Hello React  
    </h1>  
  );  
  
};  
  
export default App;
```

33. Images in React

From public

```

```

From src/assets

```
import profile from "../assets/profile.jpg";  
  
const App = () => {  
  
  return (  
    <img  
      src={profile}  
      alt="Profile"  
    >  
  );  
};
```

34. Simple React Application

Header.jsx

```
const Header = () => {  
  return (  
    <header>  
      <h1>Student Portal</h1>  
    </header>  
  );  
};  
  
export default Header;
```

Student.jsx

```
const Student = () => {

  const name = "Arun";
  const course = "React JS";

  return (
    <div>
      <h2>{name}</h2>
      <p>Course: {course}</p>
    </div>
  );
};

export default Student;
```

Footer.jsx

```
const Footer = () => {
  return (
    <footer>
      <p>Student Portal 2026</p>
    </footer>
  );
};

export default Footer;
```

App.jsx

```
import Header from "../components/Header";
import Student from "../components/Student";
import Footer from "../components/Footer";

const App = () => {

  return (
    <>
      <Header />
      <Student />
      <Footer />
    </>
  );
};
```

```
};  
  
export default App;
```

35. Complete Application Execution

When we run:

```
npm run dev
```

Conceptually:

```
Browser loads index.html  
↓  
#root  
↓  
main.jsx initializes React  
↓  
createRoot()  
↓  
<App />  
↓  
App.jsx  
↓  
Child Components  
↓  
JSX  
↓  
React renders/reconciles  
↓  
DOM changes committed  
↓  
Browser UI
```

36. Important React Terms

Term	Meaning
React	JavaScript library for building UIs
Component	Reusable UI unit
JSX	HTML-like syntax extension for JavaScript
Props	Data passed from parent to child
State	Component data that can trigger rendering updates
Real DOM	Browser's actual DOM tree
Virtual DOM	Common term for React's in-memory UI representation
Reconciliation	Process of determining required UI updates
Render	React evaluates components to determine UI
Commit	React applies required changes to the DOM
App.jsx	Common root application component file
main.jsx	Common Vite React entry file

37. Important Interview Questions

Q1. What is React?

React is an open-source JavaScript library for building component-based user interfaces.

Q2. Is React a library or framework?

React itself is primarily a **UI library**. Full applications commonly use additional libraries and tools for routing, data fetching, and other functionality.

Q3. What is a component?

A component is a reusable UI unit, commonly written as a JavaScript function that returns JSX.

Q4. Why are components important?

They improve:

- Reusability

- Maintainability
- Organization
- Readability
- Separation of concerns

Q5. What is JSX?

JSX is a syntax extension that allows HTML-like UI markup to be written inside JavaScript.

Q6. Can a browser directly execute JSX?

No. JSX normally needs to be transformed into JavaScript by tooling.

Q7. What is Virtual DOM?

It is the common term for React's in-memory representation of UI used as part of its rendering and reconciliation process.

Q8. What is the Real DOM?

It is the browser's actual Document Object Model.

Q9. What is reconciliation?

Reconciliation is React's process of determining how a new render relates to the previous render and what updates are required.

Q10. What is App.jsx?

`App.jsx` is commonly used as the root application component in Vite React projects. The filename itself is only a convention.

Q11. What is main.jsx?

`main.jsx` is commonly the application entry point that creates the React root and renders the root component.

Q12. Why should component names start with capital letters?

React uses capitalization to distinguish custom components from built-in/host elements such as `div` and `button`.

38. React Day 1 – MUST REMEMBER

React Core Flow

```
React
  ↓
Components
  ↓
JSX
  ↓
Data / State
  ↓
Render
  ↓
Reconciliation
  ↓
Commit
  ↓
Real DOM
  ↓
Browser UI
```

Project Flow

```
index.html
  ↓
main.jsx
  ↓
App.jsx
  ↓
Components
  ↓
JSX
  ↓
UI
```

Component Flow

```
Create Component
  ↓
Return JSX
  ↓
```

```
Export
↓
Import
↓
<Component />
↓
Display UI
```

39. React – 3 Phases

Phase 1: Create

```
Project Setup
↓
Dependencies
↓
Files
↓
Folders
↓
Git
```

CRA

```
npx create-react-app intro
```

Vite

```
npm create vite@latest introvite
```

Phase 2: Components

```
Single Component
↓
Multiple Components
↓
Component Composition
```

Phase 3: Update / Rendering

State / Props Change



Render



Reconciliation



Commit



UI Update

Later topics:

React Router DOM



Navigation

40. Final Day 1 Revision

If you remember only these points, remember:

React



Component-based UI

JSX



HTML-like syntax inside JavaScript

index.html



Contains #root

main.jsx



Entry point



createRoot()



```
<App />

App.jsx
↓
Root component by convention

Component
↓
Reusable UI

Props
↓
Parent → Child data

State
↓
Data that can trigger UI updates

Render
↓
React evaluates UI

Reconciliation
↓
Determines necessary changes

Commit
↓
Applies changes to DOM

Real DOM
↓
Browser's actual DOM
```

The One Flow to Memorize

```
User Action
↓
Event
↓
State Update
↓
Render
↓
Reconciliation
↓
```

Commit



Browser UI